

Process Synchronization

Part I

Sistemas de Computadores
Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

Overview

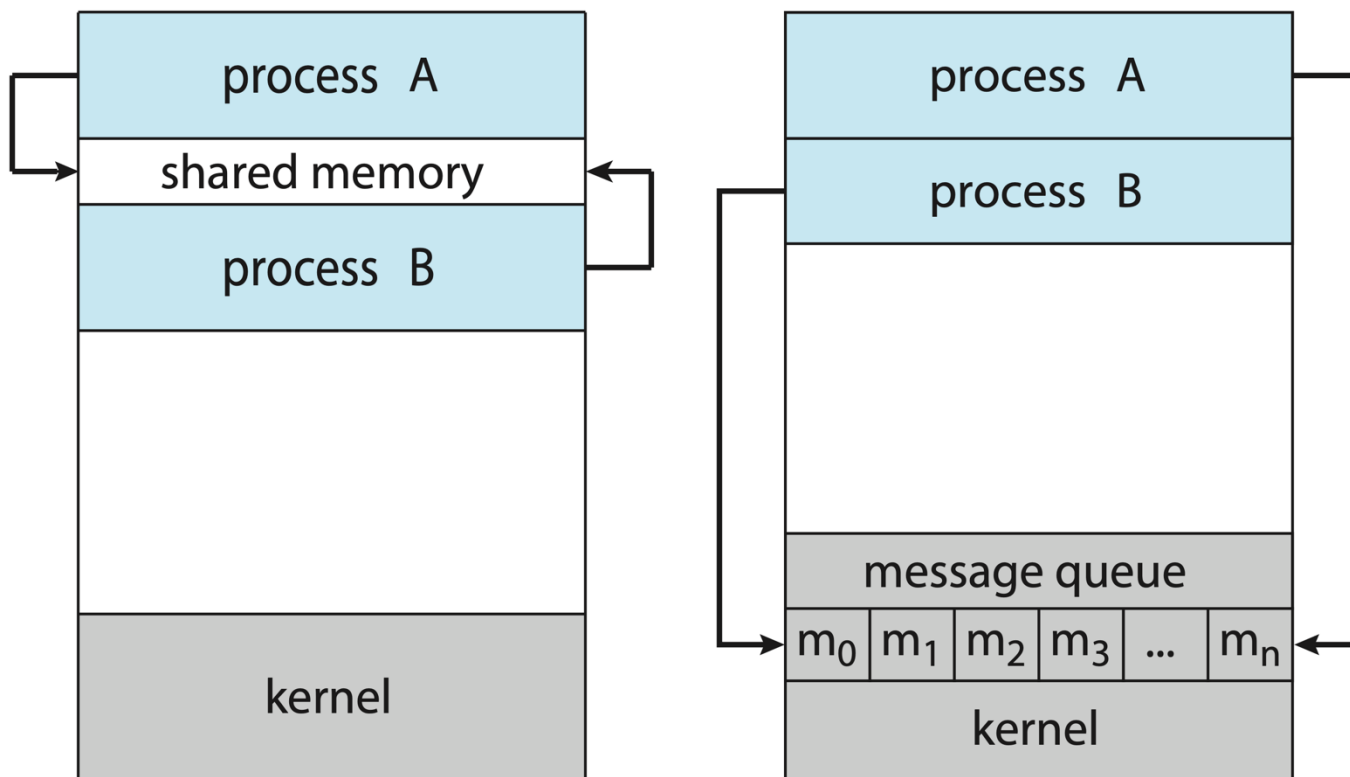
- Sharing data
- Producer-consumer problem
- Race conditions
- Semaphores

Sharing data

- **Processes executing concurrently may be independent or cooperating**
- **A process is cooperating if it can **affect or be affected** by other processes in the system**
 - This typically occurs when processes share data or communicate with each other
- **OSes support cooperating processes for several reasons:**
 - Information sharing between processes
 - Computation speedup through parallel work
 - Modularity by dividing complex applications into separate components
 - Convenience, allowing multiple processes to collaborate on a task

Sharing data

- Cooperating processes require an inter-process communication (IPC) mechanism that will allow them to exchange data
 - There are two fundamental models of IPC: **shared memory** and **message passing**



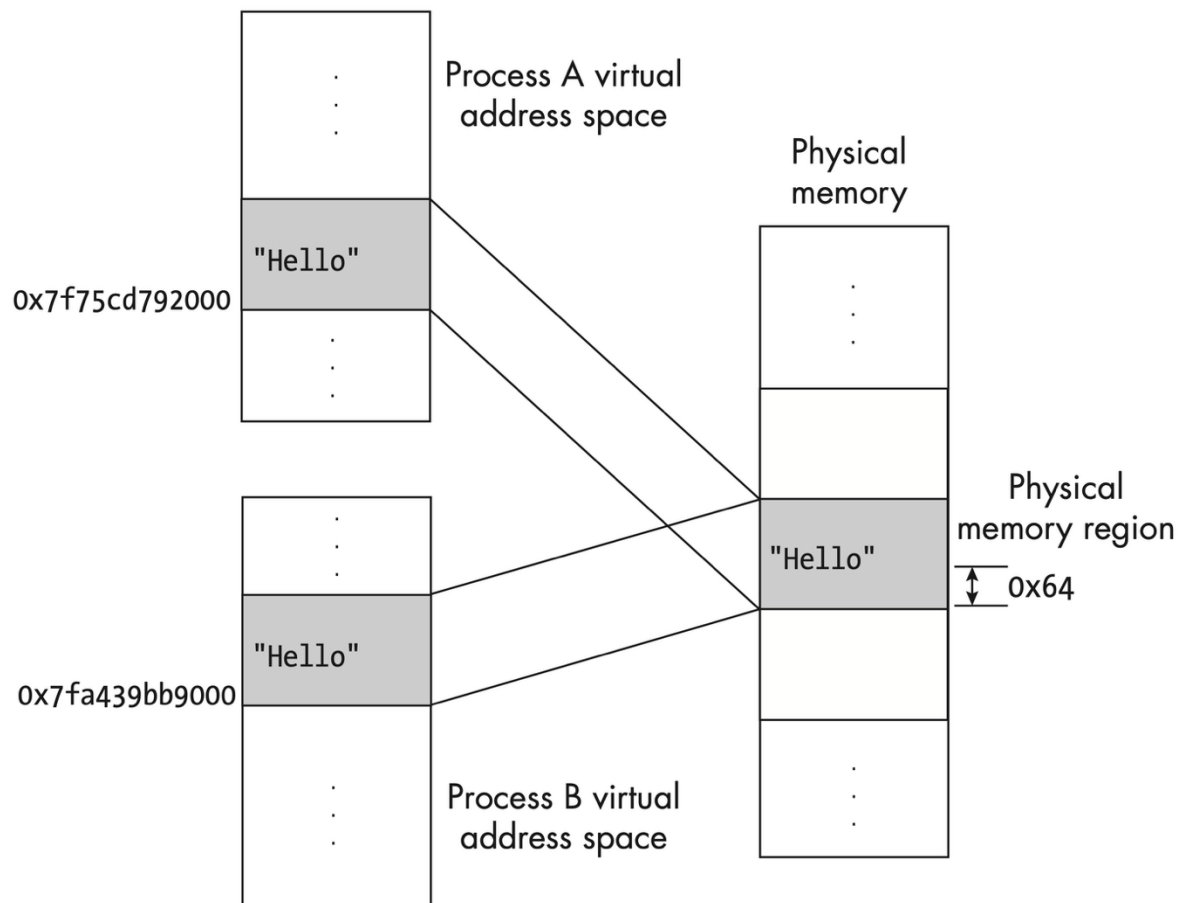
Sharing data

- **In message passing, all synchronization and communication is done by exchanging messages**
 - Is useful for exchanging smaller amounts of data
 - Is also easier to implement in a distributed system
 - Typically implemented using system calls and thus require the more time-consuming **kernel intervention**

- **Shared memory is faster and more flexible**
 - Once shared memory is established, all accesses are treated as routine memory accesses, and no assistance from the kernel is required
 - However, concurrent access to shared data may result in data inconsistency, unless proper **synchronization mechanisms** are used to ensure data integrity

Background

- Multiple virtual address spaces map the same physical frames, allowing processes to access the same physical memory region



Background

- **Concurrent execution creates situations of non-determinism**
 - The number of possible instruction interleavings grows combinatorially
 - Exhaustive testing cannot explore all possible schedules
 - Correctness must be guaranteed for all possible interleavings
- **Concurrent access to shared data may result in data inconsistency**
- **Maintaining data consistency requires mechanisms to ensure the **orderly execution** of cooperating processes**

Process synchronization

- Refers to the need for multiple processes to meet at some point in their execution to:
 1. Agree on or confirm a certain sequence of actions
 2. Ensure the coherence of shared data
- The chosen mechanism must be correct regardless of execution speeds and scheduling order
- At the same time, it is necessary that all cooperating processes **use the same synchronization mechanism** to obtain consistent results

Classic synchronization problems

- **There are several “classic” problems that represent a class of synchronization situations**
 - Critical section problem
 - Producer/Consumer problem
 - Reader/Writer problem
 - Dining philosophers
 - Sleeping barber
 - ...

- **Why study them?**
 - Once you know the “generic” solutions, you can recognize other special cases in which to apply them
 - E.g., this is just a version of the reader/writer problem

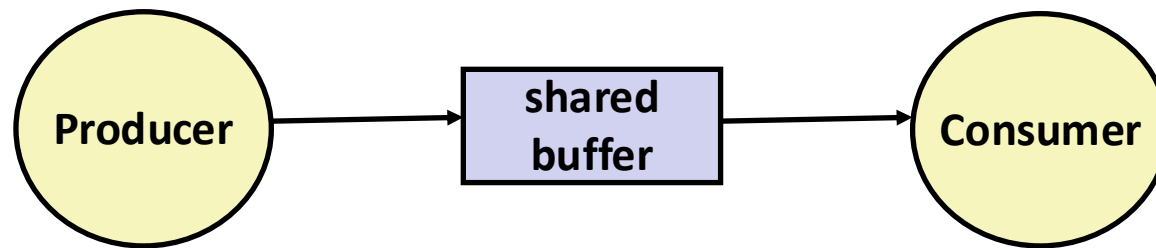
Overview

- Sharing data
- **Producer-consumer problem**
- Race conditions
- Semaphores

Producer-Consumer problem

■ To illustrate, let's consider the producer-consumer problem

- A common paradigm for cooperating processes



■ Producer

- Waits for empty slot, **inserts** item in buffer, and notifies consumer

■ Consumer

- Waits for item, **removes** it from buffer, and notifies producer

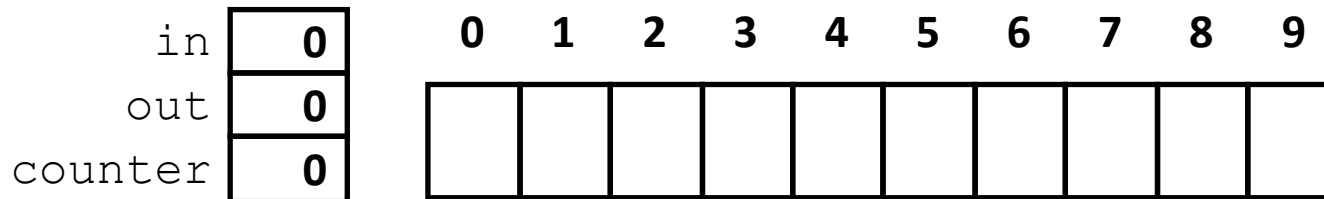
Bounded buffer implementation

- Data in a **region of memory** shared by the producer and consumer processes

```
#define BUFFER_SIZE 10

typedef struct{
    int buffer[BUFFER_SIZE];           /* circular buffer */
    int in;                           /* index of next free position */
    int out;                           /* index of first full position */
    int counter;                       /* number of items in buffer */
}shared_data;
```

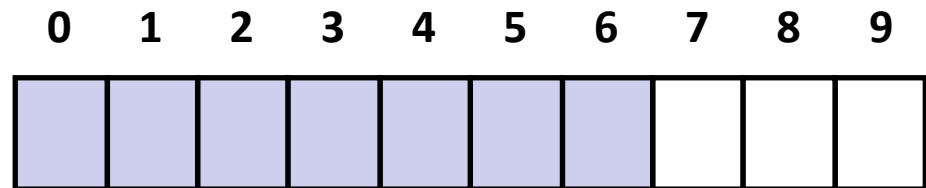
- Initially:



Circular buffer operation

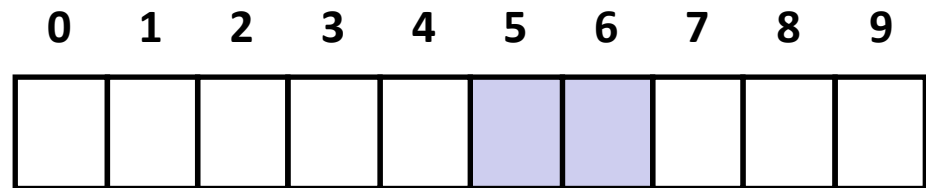
■ Insert 7 elements

in	7
out	0
counter	7



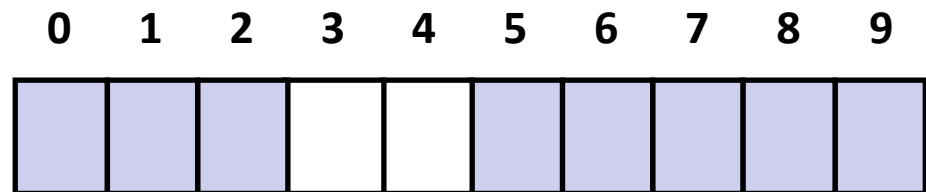
■ Remove 5 elements

in	7
out	5
counter	2



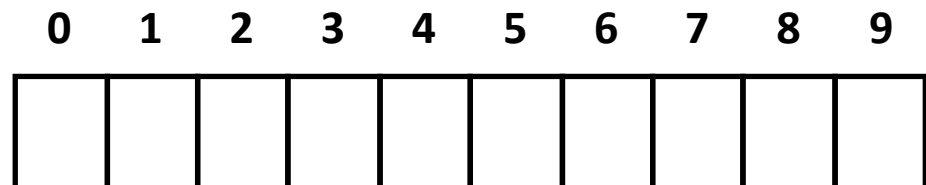
■ Insert 6 elements

in	3
out	5
counter	8



■ Remove 8 elements

in	3
out	3
counter	0



Producer-Consumer implementation

■ Producer

- Produce an item, wait for buffer space, store in buffer, notify

```
while(true) {
    /* produce an item */
    nextProduced = f();

    /* wait for buffer space */
    while(s->counter == BUFFER_SIZE);

    /* store in buffer */
    s->buffer[s->in] = nextProduced;
    s->in = (s->in + 1) % BUFFER_SIZE;

    /* notify consumer */
    s->counter++;
}
```

■ Consumer

- Wait for an item available, load it from buffer, consume it, notify

```
while(true) {
    /* wait for an item */
    while(s->counter == 0);

    /* load it from buffer */
    nextConsumed = s->buffer[s->out];

    /* consume the item */
    s->out = (s->out + 1) % BUFFER_SIZE;

    /* notify producer */
    s->counter--;
}
```

Active vs Passive waiting

■ Active (busy) waiting

- A process **repeatedly checks** to see if a condition is true
- Inefficient if used by many processes or waiting time is long
- In low-level programming, busy-waits may be desirable or required (e.g., interrupt handling)

■ Passive waiting

- Several mechanisms allow **blocking** a process until the occurrence of a given event is signaled by another process
- The blocked process is placed in a waiting queue, freeing up the CPU for other processes

Discussion on implementation

■ Data in shared memory

- `buffer[]`, `in`, `out`, `counter`

■ Busy waiting in both producer and consumer

- Ensure that consumer only access elements after they are written

■ Is this a valid/safe implementation?

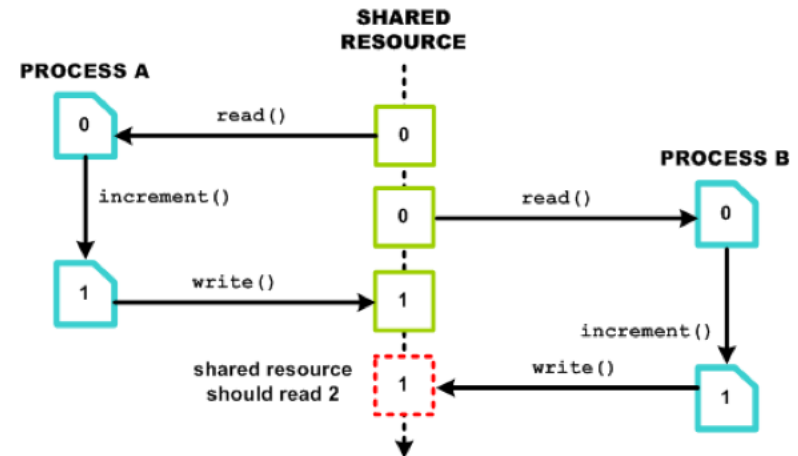
- Variable `in` only modified by producer (safe)
- Variable `out` only modified by consumer (safe)
- Variable `counter` is modified by both consumer and producer! (**unsafe**)
 - **Race condition** on the shared counter due to non-atomic updates

Overview

- Sharing data
- Producer-consumer problem
- **Race conditions**
- Semaphores

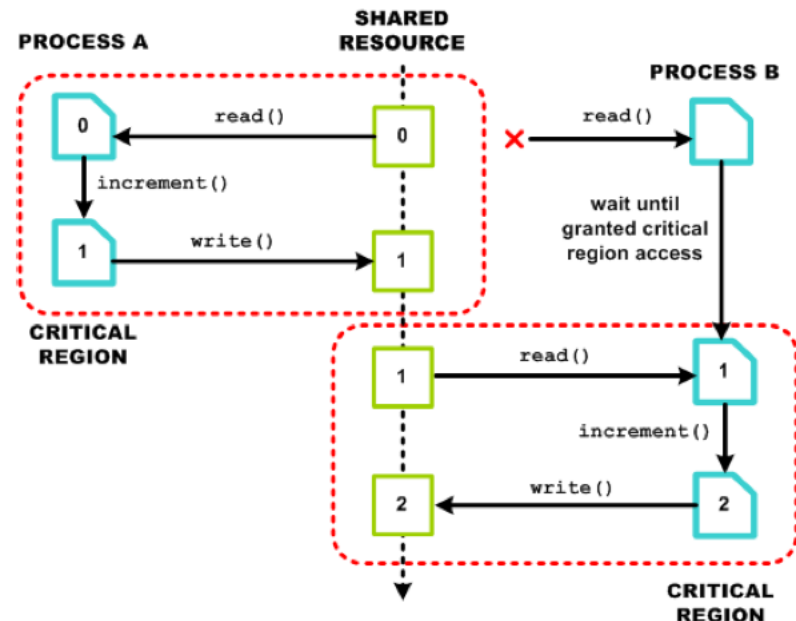
Race condition

- Concurrent access to shared data may result in **data inconsistency**
- A **race condition** exists
 - Condition of a program where its behavior depends on relative timing or interleaving of multiple processes
- Have been responsible for some serious runtime error failures over the years
 - Read about the Therac-25 radiation therapy machine



Critical section problem

- Each process has a segment of code, called a **critical section**
 - The process may be accessing — and updating — data that is shared with at least one other process
- Synchronize the execution of processes so that they can never have an invalid execution sequence
 - i.e., need to guarantee **mutually exclusive access** for each critical section



Solution to the critical section problem

■ Mutual exclusion

- If process P_i is executing in its critical section, then no other process can be executing in their critical sections

■ Progress

- If no process is in the critical section and some wish to enter, the selection of the next process cannot be postponed indefinitely

■ Bounded waiting

- A process requesting to enter its critical section must be allowed within a bounded number of entries by other processes

Software solutions

- **Several software solutions were devised to implement mutual exclusion**
 - Illustrate some of the complexities involved in designing software that addresses the requirements of mutual exclusion, progress, and bounded waiting
- **E.g., the Peterson's algorithm**
 - **Restricted to two processes** that alternate execution between their critical sections and remainder sections
 - The processes are numbered P_0 and P_1
 - For convenience, when presenting P_i , we use P_j to denote the other process; that is, j equals $1 - i$

Software solutions

■ P_i can only exit the **while** loop if one of these is true:

- `flag[j]` is false (P_j is outside the critical section)
- `turn == i` (it's P_i 's turn to enter the critical section)

```

/* i = 0 for P0, i = 1 for P1; */
/* j = 1 - I */

while (true){
    flag[i] = true;
    turn = j;

    while (flag[j] && turn == j);
    ...
    flag[i] = false;

    ...

}

```

/* State intention to enter CS */
/* Let other process go first */

/* Wait to enter CS */
/* Critical section! */
/* Leaving CS */

/* Non-critical section */

Software solutions

- **Several other software solutions were proposed**
 - e.g., Dekker's algorithm (first known correct solution), Lamport's bakery algorithm, Syzmanski's algorithm
- **All solutions are basically the same in how they work**
 - Peterson's algorithm is very representative
- **These algorithms assume sequential consistency, which modern CPUs do not guarantee without memory fences**
 - Compilers and advanced processors **frequently reorder the execution of instructions** to maximize pipelining, etc.
 - For a multitasking application with shared data, the reordering of instructions may render inconsistent or unexpected results

Hardware solutions

- **Privileged hardware instructions that allow to **atomically** – as one uninterruptible unit – to:**
 - Test and modify the content of a word (e.g., `cmpxchg` in x86)
 - Swap the contents of two words (e.g., `xchg` in x86)
 - ...
- **OS designers build higher-level software tools to solve the critical-section problem based on these hardware instructions**
- **Classic solutions:**
 - Semaphores (Edsger Dijkstra)
 - Mutexes (`pthread`s)
 - Condition variables (`pthread`s)
 - Monitors (Java)

Overview

- Sharing data
- Producer-consumer problem
- Race conditions
- **Semaphores**

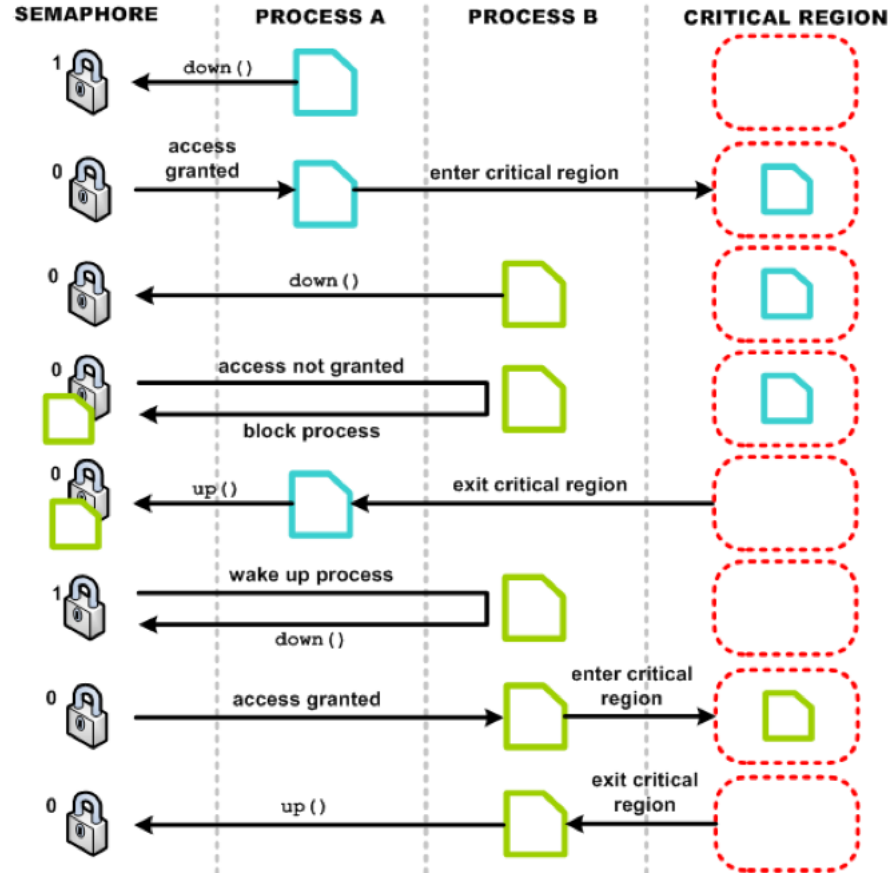
Semaphores

- **A non-negative global integer synchronization variable**
 - **Semaphore invariant: ($s \geq 0$)** (enforced by blocking semantics)
 - A list of processes waiting to acquire the semaphore
- **Manipulated by down and up operations**
 - `down(s): [ while (s == 0) wait(); s--; ]`
 - `up(s): [ s++; ]`
- **OS kernel guarantees that operations between brackets [] are **executed indivisibly** (atomically)**
 - Only one down or up operation at a time can modify `s`
 - When `while` loop in down terminates, only that down can decrement `s`

Semaphores

- Semaphores are a **generic** synchronization mechanism
 - Allows two or more processes to coordinate their actions
- Typically, processes **block until acquiring** the semaphore
 - A semaphore's value represents how many times `down()` can be called without blocking
- Its initial value (and sequence of operations) determines its use
 - 1 – mutual exclusion for accessing shared resources
 - 0 – notify another process that some condition has become true
 - ≥ 0 – counting semaphores to keep track of resource state

Example – Mutual exclusion



POSIX API

- **POSIX semaphores allow processes and threads to synchronize their actions**
- **Relevant functions for today's discussion:**

```
#include <semaphore.h>

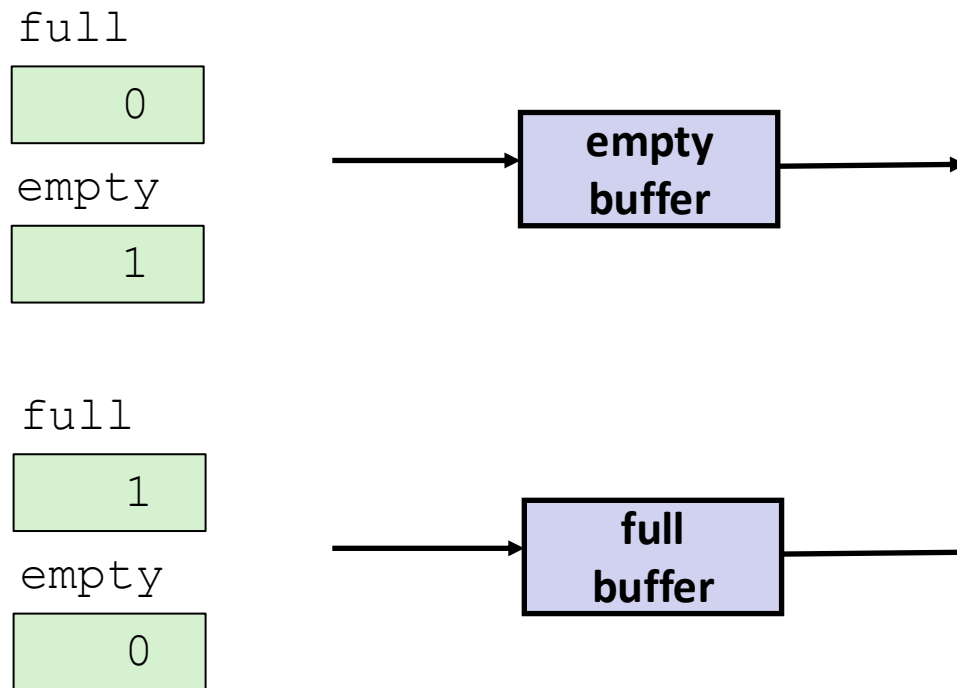
sem_t *sem_open(const char *name, int oflag,
                mode_t mode, unsigned int value);           /* s = value */

int sem_wait(sem_t *s);                                     /* down(s) */
int sem_post(sem_t *s);                                     /* up(s) */
```

Producer-Consumer on 1-element buffer

■ Maintain two semaphores:

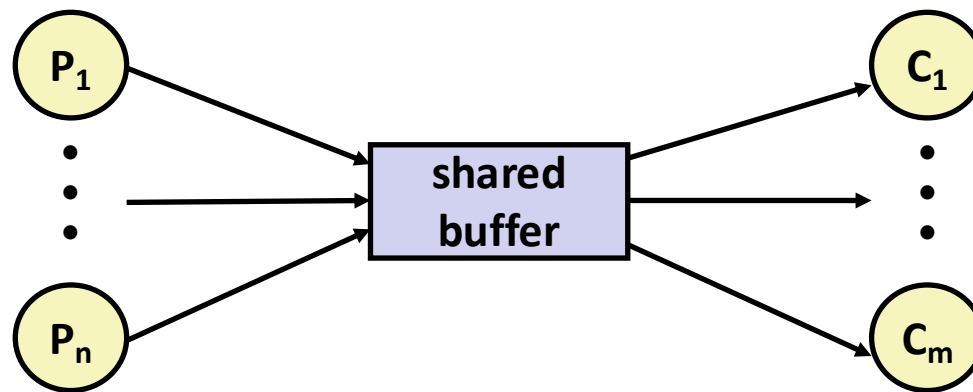
- `full` and `empty`



Why 2 semaphores for 1-element buffer?

■ Consider multiple producers and multiple consumers

- Producers will contend with each to get `empty`
- Consumers will contend with each other to get `full`



n x Producers

```
sem_wait(empty);  
s->buffer = nextProduced;  
sem_post(full);
```

m x Consumers

```
sem_wait(full);  
nextConsumed = s->buffer;  
sem_post(empty);
```

Producer-Consumer on 1-element buffer

```
typedef struct{
    int buffer;                /* 1-element buffer */
}shared_data;

sem_t *full;                  /* block producer on full buffer */
sem_t *empty;                  /* block consumer on empty buffer */

full = sem_open("/full", O_CREAT, 0644, 0);
empty = sem_open("/empty", O_CREAT, 0644, 1);
```

/* Producer */

```
while(true){
    /* produce an item */
    nextProduced = f();

    /* wait for buffer space */
    sem_wait(empty);
    /* store in buffer */
    s->buffer = nextProduced;

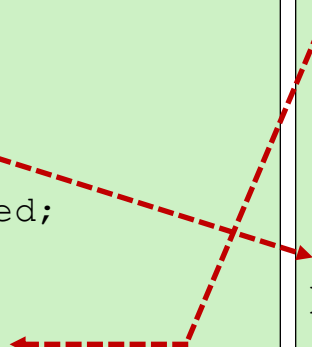
    /* notify consumer */
    sem_post(full);
}
```

/* Consumer */

```
while(true){
    /* wait for an item */
    sem_wait(full);

    /* load it from buffer */
    nextConsumed = s->buffer;

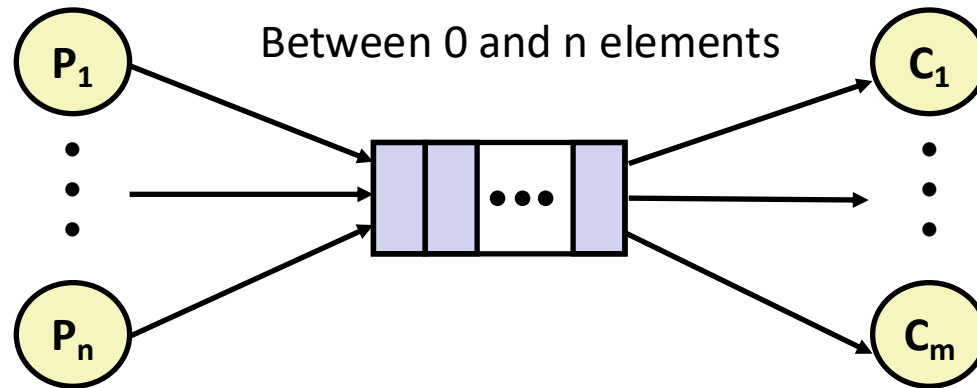
    /* notify producer */
    sem_post(empty);
}
```



Producer-Consumer on an n -element buffer

■ Requires **mutual exclusion** and two **counting semaphores**

- `mutex`: enforces mutually exclusive access to the buffer and counters
- `slots`: counts the available slots in the buffer
- `items`: counts the available items in the buffer



Declarations

```
#define N 10                                /* Maximum number of slots */

typedef struct {
    int buffer[N];                          /* Buffer array */
    int in;                                /* Index of next free position */
    int out;                               /* Index of first full position */
} sbuf_t;

sem_t *mutex;                            /* Protects accesses to buffer */
sem_t *slots;                            /* Counts available slots */
sem_t *items;                           /* Counts available items */

void sbuf_init(sbuf_t *sp);
void sbuf_insert(sbuf_t *sp, int item);
int sbuf_remove(sbuf_t *sp);
```

Initialization and clearing

```
/* Initializes semaphores */  
mutex = sem_open("/mutex", O_CREAT, 0644, 1);  
slots = sem_open("/slots", O_CREAT, 0644, N);  
items = sem_open("/items", O_CREAT, 0644, 0);  
  
/* Create an empty, bounded, shared FIFO buffer with N slots */  
void sbuf_init(sbuf_t *sp){  
    memset(sp->buffer, 0, N*sizeof(int));  
    sp->in = sp->out = 0;  
}
```

Inserting an item

```
/* Insert item onto the rear of shared buffer sp */  
void sbuf_insert(sbuf_t *sp, int item){  
    /* Wait for available slot */  
    sem_wait(slots);  
  
    /* Lock the buffer */  
    sem_wait(mutex);  
  
    /* store in buffer */  
    sp->buffer[sp->in] = item;  
    sp->in = (sp->in + 1) % N;  
  
    /* Unlock the buffer */  
    sem_post(mutex);  
  
    /* Announce available item */  
    sem_post(items);  
}
```



Critical section

Removing an item

```
/* Remove and return the first item from buffer sp */  
int sbuf_remove(sbuf_t *sp) {  
    int item;  
    /* Wait for available item */  
    sem_wait(items);  
  
    /* Lock the buffer */  
    sem_wait(mutex);  
  
    /* load it from buffer */  
    item = sp->buffer[sp->out];  
    /* consume the item */  
    sp->out = (sp->out + 1) % N;  
  
    /* Unlock the buffer */  
    sem_post(mutex);  
  
    /* Announce available slot */  
    sem_post(slots);  
  
    return item;  
}
```



Critical section

Demonstration

- See program `producer-consumer.c` in Moodle
- 10-entry shared circular buffer
- 5 producers
 - Producer i generates numbers from $20*i$ to $20*(i + 1)$
 - Puts them in buffer
- 5 consumers
 - Consumer i retrieves 20 elements from buffer
 - Prints them

Summary

- **Correct concurrent programs require a precise mental model of how processes share and access data**
- **Any modification of shared data must be performed under controlled, mutually exclusive access**
- **Synchronization mechanisms coordinate process execution to ensure correctness**
- **Semaphores are a fundamental abstraction for implementing synchronization and resource management**